

Contents

Apuntes de Python 3.12 (listas, funciones, NumPy, diccionarios, pandas) — Guía para la Práctica 3	1
Sección A — Listas	1
Sección B — Funciones y paquetes	4
Sección C — NumPy	7
Sección D — Diccionarios	10
Sección E — pandas	12
Sección F — Lógica y control de flujo	15
Checklist rápido de conceptos por ejercicio	16
Ejercicios resueltos	16
Celda de verificación del entorno	16
Ejercicio 1 — Catálogo de productos como lista	17
Ejercicio 2 — Inventario multialmacén con listas anidadas	17
Ejercicio 3 — Manipulación destructiva y copias de listas	18
Ejercicio 4 — Cola de pedidos pendientes	19
Ejercicio 5 — Funciones propias para el cálculo de importes	19
Ejercicio 6 — Normalización de códigos SKU con métodos	20
Ejercicio 7 — Paquetes, importaciones y módulo propio	21
Ejercicio 8 — Del cálculo con listas al cálculo vectorizado	22
Ejercicio 9 — Subsetting con máscaras booleanas	22
Ejercicio 10 — Matriz de ventas por trimestre y región	23
Ejercicio 11 — Estadística descriptiva sobre telemetría simulada	23
Ejercicio 12 — Diccionarios simples y anidados	24
Ejercicio 13 — De diccionario a DataFrame y lectura de CSV	25
Ejercicio 14 — Selección con corchetes, loc e iloc	26
Ejercicio 15 — Columnas calculadas, agregación y exportación	26
Ejercicio 16 — Operadores de comparación y booleanos	27

Apuntes de Python 3.12 (listas, funciones, NumPy, diccionarios, pandas) — Guía para la Práctica 3

Formato de cada apunte: **Concepto** → **Explicación** → **Sintaxis básica** → **Ejemplo práctico**. Los ejemplos están contruidos sobre los mismos datos de la práctica (catálogo de productos, inventario, ventas, empleados, sensores) para que los puedas adaptar directamente.

Sección A — Listas

1. Indexación y slicing con paso

Explicación: Una lista se indexa desde 0. El *slicing* `lista[inicio:fin:paso]` extrae un tramo; fin no se incluye. Un paso de 2 salta de dos en dos (útil para separar elementos alternados). Los índices negativos cuentan desde el final (-1 es el último elemento).

Sintaxis básica:

```
lista[inicio:fin]           # tramo normal
lista[inicio:fin:paso]     # con salto
lista[-n:]                 # los últimos n elementos
```

Ejemplo práctico:

```

catalogo = ["Auriculares BT", 59.90, "Smartwatch S2", 149.00, "Tablet 10", 219.00]

nombres = catalogo[0:2]      # posiciones pares: los nombres
precios = catalogo[1:2]      # posiciones impares: los precios
ultimos_tres = catalogo[-6:] # últimos elementos con índice negativo

print(len(catalogo))          # 6
print(nombres)                 # ['Auriculares BT', 'Smartwatch S2', 'Tablet 10']
print(round(sum(precios) / len(precios), 2))

```

2. .index() para localizar posiciones sin “números mágicos”

Explicación: .index(valor) devuelve la posición de la primera aparición de un valor en la lista. Combinarlo con la posición relativa evita escribir índices a mano, lo que hace el código más robusto ante cambios en los datos.

Sintaxis básica:

```
posicion = lista.index(valor)
```

Ejemplo práctico:

```

nombres = ["Auriculares BT", "Smartwatch S2", "Tablet 10", "Cargador USB-C", "Cafetera Expres", "Robot Aspirador"]
precios = [59.90, 149.00, 219.00, 18.50, 189.00, 279.00]

pos = nombres.index("Robot Aspirador")
print(precios[pos]) # 279.0

```

3. Listas anidadas (lista de listas)

Explicación: Una lista puede contener otras listas como elementos. Para acceder a un valor concreto se encadena la indexación: primero la fila, después la columna (matriz[filas][columna]).

Sintaxis básica:

```

matriz = [[a, b, c], [d, e, f]]
matriz[filas][columna]

```

Ejemplo práctico:

```

inventario = [
    ["ALM-NORTE", "Robot Aspirador", 34, 201.50],
    ["ALM-NORTE", "Monitor 27", 58, 142.00],
    ["ALM-SUR", "Silla Ergonomica", 12, 178.90],
    ["ALM-SUR", "Robot Aspirador", 7, 201.50],
]

print(inventario[3][1]) # "Robot Aspirador" (cuarta fila, columna producto)

valor_stock = []
for fila in inventario:
    almacen, producto, unidades, coste = fila
    valor_stock.append([producto, round(unidades * coste, 2)])

valor_total = sum(u * c for _, u, c, in [(f[1], f[2], f[3]) for f in inventario])

```

Nota sobre slicing como filtro: un *slicing* solo funciona si las filas que buscas están contiguas en la lista. Si un almacén deja de estar seguido de otro, el slicing dejaría fuera o incluiría filas equivocadas; para filtrar de forma robusta hace falta recorrer y comprobar una condición (o usar pandas más adelante), no depender del orden físico.

4. Referencias vs copias de listas (=, list(), [:], id())

Explicación: En Python, `b = a` **no copia** la lista: `b` pasa a apuntar al mismo objeto en memoria que `a`, así que modificar `b` también modifica `a`. Para obtener una copia independiente hay que usar `list(a)`, `a[:]` o `a.copy()`. `id(objeto)` devuelve la dirección de memoria y permite comprobar si dos variables apuntan al mismo objeto.

Sintaxis básica:

```
b = a          # misma referencia (peligroso)
b = list(a)    # copia independiente
b = a[:]       # copia independiente (slicing completo)
id(a) == id(b) # False si son copias distintas
```

Ejemplo práctico:

```
precios_originales = [59.90, 149.00, 219.00, 18.50, 189.00, 279.00]

precios_rebajados = precios_originales      # misma lista
precios_rebajados[0] = 49.90
print(precios_originales[0])                # 49.90 -> ¡también cambió el original!

precios_originales[0] = 59.90               # restaurar
copia_a = list(precios_originales)
copia_b = precios_originales[:]
copia_a[0] = 0
print(precios_originales[0])                # 59.90, no cambió

print(id(precios_originales) == id(copia_a)) # False
```

5. Métodos que modifican en sitio: .append(), .extend(), .insert(), .pop(), .remove(), del, .sort(), .reverse()

Explicación: Estos métodos **modifican la lista original** y devuelven `None` (no se puede encadenar `resultado = lista.sort()`). `.append(x)` añade un elemento al final; `.extend(iterable)` añade varios; `.insert(pos, x)` inserta en una posición concreta; `.pop(pos)` elimina y **devuelve** el elemento en esa posición (por defecto el último); `del lista[pos]` elimina sin devolver nada; `.sort()` ordena en sitio; `sorted(lista)` en cambio **no** modifica la original, devuelve una lista nueva ordenada.

Sintaxis básica:

```
lista.append(x)
lista.extend([x, y])
lista.insert(pos, x)
valor = lista.pop(pos)
del lista[pos]
lista.sort(reverse=True)    # modifica en sitio
nueva = sorted(lista)      # no modifica, devuelve copia ordenada
```

Ejemplo práctico:

```
cola = ["PED-10021", "PED-10007", "PED-10044", "PED-10012", "PED-10033", "PED-10008"]

cola_invertida = cola[::-1]          # slicing con paso negativo, no modifica cola

cola_ordenada = sorted(cola)         # copia ordenada, cola sigue igual
cola.sort(reverse=True)             # ahora sí, modifica cola en sitio

atendidos = [cola.pop(0), cola.pop(0)]
cola.insert(0, "PED-99999")

print(f"Atendidos: {len(atendidos)}, Pendientes: {len(cola)}")
```

Sección B — Funciones y paquetes

6. Funciones con parámetros por defecto

Explicación: Un parámetro con valor por defecto (param=valor) es opcional al llamar a la función; si no se indica, se usa ese valor. Los parámetros con valor por defecto deben ir **después** de los obligatorios.

Sintaxis básica:

```
def funcion(obligatorio, opcional=valor_por_defecto):
    return resultado
```

Ejemplo práctico:

```
def calcular_importe(unidades, precio_unitario, descuento_pct=0):
    """Devuelve el importe neto de una línea de pedido, redondeado a 2 decimales."""
    bruto = unidades * precio_unitario
    return round(bruto * (1 - descuento_pct / 100), 2)

print(calcular_importe(3, 59.90, 10))    # 161.73
print(calcular_importe(1, 279.00))      # sin descuento, usa el valor por defecto 0
```

7. Funciones que devuelven varios valores (tuplas) y desempaqueado

Explicación: Si una función hace return a, b, c, en realidad devuelve una **tupla** de tres elementos. Al llamarla se puede “desempaquetar” directamente en varias variables separadas por comas.

Sintaxis básica:

```
def funcion():
    return valor1, valor2, valor3

a, b, c = funcion()
```

Ejemplo práctico:

```
def resumen_pedido(unidades, precio_unitario, descuento_pct=0):
    """Devuelve (importe_bruto, ahorro, importe_netto)."""
    bruto = round(unidades * precio_unitario, 2)
    neto = calcular_importe(unidades, precio_unitario, descuento_pct)
```

```

    ahorro = round(bruto - neto, 2)
    return bruto, ahorro, neto

# llamada con argumentos por palabra clave (keyword arguments)
bruto, ahorro, neto = resumen_pedido(unidades=2, precio_unitario=149.00, descuento_pct=5)
print(bruto, ahorro, neto)

```

8. Docstrings y help()

Explicación: Un *docstring* es una cadena de texto justo debajo de `def`, entre comillas triples, que documenta qué hace la función. `help(funcion)` la muestra en pantalla.

Sintaxis básica:

```

def funcion(param):
    """Descripción breve de lo que hace la función."""
    ...

```

Ejemplo práctico:

```

help(calcular_importe)
# Muestra: calcular_importe(unidades, precio_unitario, descuento_pct=0)
#         Devuelve el importe neto de una línea de pedido, redondeado a 2 decimales.

```

9. Funciones integradas sobre listas: max(), min(), sorted(), len()

Explicación: Funciones que operan directamente sobre cualquier lista de números: `max/min` devuelven el mayor/menor valor, `sorted` una copia ordenada, `len` el número de elementos.

Sintaxis básica:

```

max(lista)
min(lista)
sorted(lista)           # ascendente
sorted(lista, reverse=True)
len(lista)

```

Ejemplo práctico:

```

pedidos = [(3, 59.90, 10), (1, 279.00, 0), (5, 18.50, 20), (2, 149.00, 5), (4, 89.90, 15)]
importes = [calcular_importe(u, p, d) for u, p, d in pedidos]

print(max(importes), min(importes), sorted(importes), len(importes))

```

10. Métodos de cadenas: .strip(), .upper(), .lower(), .replace(), .split(), .join(), .count(), .index()

Explicación: Todos los métodos de `str` **devuelven una cadena nueva**, nunca modifican la original (las cadenas son inmutables en Python). `.strip()` elimina espacios al principio/final; `.split(sep)` trocea una cadena en una lista según un separador; `sep.join(lista)` hace lo contrario, une una lista de cadenas con un separador.

Sintaxis básica:

```

texto.strip()
texto.upper()
texto.replace(viejo, nuevo)
texto.split(separador)      # str -> list
separador.join(lista)       # list -> str
lista.count(valor)
lista.index(valor)

```

Ejemplo práctico:

```

skus_crudos = [" elc-0012-es ", "H0G-0045-ES", "dep-0003-pt ", "ofi-0021-es"]

skus_limpios = [s.strip().upper() for s in skus_crudos]
print(skus_limpios.count("ELC-0012-ES"))
print(skus_limpios.index("OFI-0021-ES"))

partes = [s.split("-") for s in skus_limpios]  # [['ELC', '0012', 'ES'], ...]

skus_es = [s.replace("-", ".") for s in skus_limpios if s.endswith("ES")]
print(" | ".join(skus_es))

```

11. Importar paquetes: import, from ... import ..., alias

Explicación: `import paquete` carga el paquete entero, accediendo a sus funciones con `paquete.funcion()`. `from paquete import funcion` importa solo lo necesario, sin prefijo. `import paquete as alias` (convención: `numpy` as `np`, `pandas` as `pd`) acorta el nombre.

Sintaxis básica:

```

import math
from math import pi, ceil
import numpy as np
import pandas as pd

```

Ejemplo práctico:

```

import math
print(math.sqrt(2809))      # 53.0

from math import pi, ceil
palets = ceil(1375 / 24)    # palés completos necesarios

import numpy as np
import pandas as pd
print(np.__version__, pd.__version__)

```

12. Módulo random con semilla fija (reproducibilidad)

Explicación: `random.seed(n)` fija el “punto de partida” del generador de números pseudoaleatorios. Con la misma semilla, la secuencia de números generados es siempre idéntica, lo que permite que dos personas obtengan exactamente el mismo resultado “aleatorio” y puedan comparar o depurar el análisis.

Sintaxis básica:

```
import random
random.seed(n)
random.randint(a, b)
```

Ejemplo práctico:

```
import random
random.seed(42)
valoraciones = [random.randint(1, 5) for _ in range(10)]
print(valoraciones)  # siempre el mismo resultado con seed(42)
```

13. Módulo propio e importación con `sys.path`

Explicación: Se puede crear un fichero `.py` con funciones (un módulo) y, desde un notebook situado en otra carpeta, indicarle a Python dónde buscarlo añadiendo esa ruta a `sys.path` antes de importarlo con `from modulo import funcion`.

Sintaxis básica:

```
import sys
sys.path.append("../src")
from utilidades import funcion
```

Ejemplo práctico:

```
# src/utilidades.py
"""Funciones auxiliares de la Práctica 3."""

def calcular_importe(unidades, precio_unitario, descuento_pct=0):
    bruto = unidades * precio_unitario
    return round(bruto * (1 - descuento_pct / 100), 2)

def clasificar_ticket(importe):
    if importe < 100:
        return "Bajo"
    elif importe < 400:
        return "Medio"
    return "Alto"

# En el notebook, dentro de notebooks/
import sys
sys.path.append("../src")
from utilidades import calcular_importe, clasificar_ticket

print(calcular_importe(4, 89.90, 15))  # 305.66
print(clasificar_ticket(305.66))      # "Medio"
```

Sección C — NumPy

14. Arrays NumPy vs listas: operaciones vectorizadas

Explicación: Con listas de Python, el operador `*` **no** multiplica elemento a elemento (repite o falla); con arrays de NumPy, las operaciones aritméticas se aplican automáticamente a **cada elemento** (“vectorizadas”), sin necesidad de bucles explícitos, y son mucho más rápidas.

Sintaxis básica:

```
import numpy as np
arr = np.array(lista)
arr.dtype      # tipo de dato de los elementos
arr.shape      # dimensiones
resultado = arr1 * arr2  # elemento a elemento
```

Ejemplo práctico:

```
import numpy as np

unidades = [3, 1, 5, 2, 4]
precios = [59.90, 279.00, 18.50, 149.00, 89.90]

# unidades * precios -> con listas de Python esto NO multiplica elemento a elemento

np_unidades = np.array(unidades)
np_precios = np.array(precios)
print(np_unidades.dtype, np_unidades.shape)

importes = np_unidades * np_precios
print(importes, importes.sum())

importes_con_recargo = importes * 1.04  # recargo del 4% a todos a la vez
```

Un array con tipos mixtos fuerza la homogeneidad (todos los elementos pasan al tipo más general, normalmente texto):

```
mixto = np.array([1, "dos", 3.0, True])
print(mixto.dtype)  # '<U...>' (todo se convierte a string)
```

15. Máscaras booleanas y np.where()

Explicación: Comparar un array con un valor (`array > 100`) devuelve un array del mismo tamaño con True/False en cada posición: una **máscara booleana**. Usar esa máscara entre corchetes filtra el array, devolviendo solo los elementos donde es True. `mascara.sum()` cuenta los True (porque True vale 1 y False vale 0). `np.where(mascara)` devuelve las posiciones donde la condición se cumple.

Sintaxis básica:

```
mascara = array > valor
array[mascara]
mascara.sum()
np.where(mascara)
```

Ejemplo práctico:

```
caros = np_precios > 100
print(caros)                # [False True False True ...]
print(np_precios[caros])    # solo los precios > 100
print(caros.sum())          # cuántos hay
print(importes[caros].sum()) # su facturación

posiciones = np.where(caros)
```

16. Combinar condiciones en NumPy: &, | (no and/or)

Explicación: Con arrays de NumPy, los operadores lógicos de Python and/or **no funcionan** elemento a elemento y lanzan error; hay que usar & (y) y | (o), con cada condición **obligatoriamente entre paréntesis**, porque &| tienen mayor prioridad que >/< y sin paréntesis el resultado sería incorrecto o un error.

Sintaxis básica:

```
(condicion1) & (condicion2)
(condicion1) | (condicion2)
```

Ejemplo práctico:

```
seleccion = (np_precios > 100) & (np_unidades >= 2)
print(importes[seleccion])
```

*# temp > 70 and vib > 3 -> ValueError: la verdad de un array con más
de un elemento es ambigua; and/or esperan un único booleano, no un array*

17. Arrays 2D: indexación, slicing, axis, np.vstack(), np.argmax()

Explicación: Un array 2D tiene filas y columnas; se indexa como array[filas, columna]. axis=0 opera “hacia abajo” (una suma por columna, es decir, por cada región); axis=1 opera “hacia la derecha” (una suma por fila, por cada trimestre). np.vstack() apila arrays añadiendo filas nuevas. np.argmax() devuelve la posición del valor máximo.

Sintaxis básica:

```
array2d[filas, columna]
array2d[filas, :]        # fila completa
array2d[:, columna]      # columna completa
array2d.sum(axis=0)       # total por columna
array2d.sum(axis=1)       # total por fila
np.vstack([array2d, filas_nuevas])
np.argmax(array1d)
```

Ejemplo práctico:

```
ventas_2d = np.array([
    [15200.50, 11800.25, 12950.00, 9870.40, 13400.10],
    [16840.75, 12310.60, 13480.90, 11020.35, 14100.80],
    [12535.72, 10663.06, 11221.59, 12673.08, 12334.38],
])

print(ventas_2d.shape, ventas_2d.ndim, ventas_2d.size)

filas_t2 = ventas_2d[1, :]
columna_centro = ventas_2d[:, 4]
valor_t3_este = ventas_2d[2, 2]

submatriz = ventas_2d[0:2, 0:3]

total_por_trimestre = ventas_2d.sum(axis=1)
total_por_region = ventas_2d.sum(axis=0)

prevision_t4 = ventas_2d[2, :] * 1.06
```

```
ventas_ampliada = np.vstack([ventas_2d, prevision_t4])

regiones = ["Norte", "Sur", "Este", "Oeste", "Centro"]
region_lider = regiones[np.argmax(total_por_region)]
```

18. Generación aleatoria reproducible y estadística descriptiva

Explicación: `np.random.default_rng(semilla)` crea un generador reproducible (versión moderna recomendada frente a `np.random.seed()`). Funciones estadísticas clave: `.mean()`, `.median()` (o `np.median()`), `.std()`, `.min()`, `.max()`, `np.percentile(array, p)`. `np.corrcoef(a, b)` calcula la matriz de correlación entre dos variables.

Sintaxis básica:

```
rng = np.random.default_rng(semilla)
rng.normal(media, desviacion, tamaño)
array.mean(); array.std(); np.median(array)
np.percentile(array, [25, 50, 75])
np.corrcoef(a, b)
```

Ejemplo práctico:

```
rng = np.random.default_rng(2024)
temperatura = rng.normal(68, 6.5, 500)
vibracion = np.abs(rng.normal(2.4, 0.8, 500))

media = temperatura.mean()
mediana = np.median(temperatura)
desviacion = temperatura.std()

percentiles = np.percentile(vibracion, [25, 50, 75, 95])

matriz_2d = np.column_stack([temperatura, vibracion])
print(matriz_2d.shape)  # (500, 2)

correlacion = np.corrcoef(temperatura, vibracion)

umbral = media + 2 * desviacion
n_superan = (temperatura > umbral).sum()
pct_superan = round(100 * n_superan / len(temperatura), 2)
```

Fijar la semilla (`default_rng(2024)`) garantiza que cualquier persona que ejecute el mismo código obtenga exactamente los mismos “datos aleatorios”, lo cual es imprescindible para que un análisis sea reproducible y verificable por un tercero.

Sección D — Diccionarios

19. Diccionarios: creación, acceso, modificación, recorrido

Explicación: Un diccionario almacena pares clave: valor. Se accede por clave (no por posición), se añade o actualiza asignando `dict[clave] = valor`, se elimina con `del dict[clave]`, y se comprueba si una clave existe con `in`. `.items()` permite recorrer pares clave-valor a la vez.

Sintaxis básica:

```
d = {"clave1": valor1, "clave2": valor2}
d["clave"]          # acceso
d["nueva_clave"] = valor  # añadir o actualizar
del d["clave"]
"clave" in d
for clave, valor in d.items():
    ...
```

Ejemplo práctico:

```
tarifas_envio = {"Norte": 4.95, "Sur": 5.50, "Este": 5.20, "Oeste": 6.10, "Centro": 3.90}

print(tarifas_envio["Este"])

tarifas_envio["Insular"] = 9.80
tarifas_envio["Oeste"] = 5.95

del tarifas_envio["Sur"]
print("Sur" in tarifas_envio)  # False

for region, tarifa in tarifas_envio.items():
    print(f"{region} -> {tarifa} €")
```

20. Diccionarios anidados

Explicación: El valor de una clave puede ser, a su vez, otro diccionario. Se accede encadenando claves: `d[clave_externa][clave_interna]`. Recorrer la estructura completa requiere un bucle anidado (bucle sobre las claves externas, y dentro otro sobre `.items()` del diccionario interno).

Sintaxis básica:

```
d = {
    "external": {"internal1": valor1, "internal2": valor2},
}
d["external"]["internal1"]
```

Ejemplo práctico:

```
red_logistica = {
    "Norte": {"almacen": "ALM-NORTE", "tarifa": 4.95, "entrega_h": 24},
    "Este": {"almacen": "ALM-ESTE", "tarifa": 5.20, "entrega_h": 48},
    "Centro": {"almacen": "ALM-CENTRO", "tarifa": 3.90, "entrega_h": 24},
}

print(red_logistica["Este"]["entrega_h"])  # 48

red_logistica["Oeste"] = {"almacen": "ALM-OESTE", "tarifa": 5.95, "entrega_h": 24}

for region, datos in red_logistica.items():
    print(f"{region}:")
    for campo, valor in datos.items():
        print(f"    {campo} -> {valor}")
```

Un diccionario conviene frente a una lista cuando el acceso natural es “por nombre” en lugar de “por posición” (por ejemplo, buscar la tarifa de “Este” es más claro y seguro que recordar que ocupa la posición 2); además, la clave aporta significado por sí misma, mientras que un índice

numérico no dice nada sobre el dato al que apunta.

Sección E — pandas

21. Crear un DataFrame desde un diccionario

Explicación: `pd.DataFrame(diccionario)` convierte un diccionario donde cada clave es una columna y cada valor una lista de datos, en una tabla (DataFrame). Por defecto pandas crea un índice numérico automático (0, 1, 2...).

Sintaxis básica:

```
import pandas as pd
df = pd.DataFrame({"col1": [...], "col2": [...]}
df.set_index("columna")  # usar una columna como índice
```

Ejemplo práctico:

```
import pandas as pd

almacenes = pd.DataFrame({
    "codigo": ["ALM-NORTE", "ALM-SUR", "ALM-ESTE", "ALM-OESTE", "ALM-CENTRO"],
    "ciudad": ["Bilbao", "Sevilla", "Valencia", "Vigo", "Madrid"],
    "m2": [4200, 3100, 3800, 2600, 6500],
    "operarios": [48, 31, 39, 22, 74],
})

print(almacenes)  # índice 0..4
almacenes_idx = almacenes.set_index("codigo")
print(almacenes_idx)
```

22. Leer CSV y explorar un DataFrame

Explicación: `pd.read_csv(ruta)` carga un fichero CSV como DataFrame; `index_col="columna"` fija esa columna como índice desde la carga. `.head(n)/.tail(n)` muestran las primeras/últimas filas, `.shape` da (filas, columnas), `.info()` resume tipos y nulos por columna, `.isna().sum()` cuenta valores ausentes por columna.

Sintaxis básica:

```
df = pd.read_csv("ruta/relativa.csv", index_col="columna")
df.head(n); df.tail(n); df.shape
df.info()
df.isna().sum()
```

Ejemplo práctico:

```
ventas = pd.read_csv("../data/ventas_retail.csv")
empleados = pd.read_csv("../data/empleados.csv", index_col="id_empleado")
sensores = pd.read_csv("../data/sensores_planta.csv")

print(ventas.head(8))
print(ventas.tail(3))
print(ventas.shape)  # (420, ...)
```

```
ventas.info()
print(ventas.isna().sum()) # nulos concentrados en 'canal' y 'satisfaccion'
```

23. Selección de columnas y filas: corchetes, loc, iloc

Explicación: `df["col"]` devuelve una **Serie** (1 dimensión); `df[["col"]]` devuelve un **DataFrame** (1 columna, pero en formato tabla). `df[["col1", "col2"]]` selecciona varias columnas. `.loc[]` selecciona **por etiqueta** (nombre de índice/columna); `.iloc[]` selecciona **por posición** numérica, igual que con listas. Ambos aceptan filas y columnas a la vez: `.loc[filas, columnas]`.

Sintaxis básica:

```
df["columna"] # Serie
df[["columna"]] # DataFrame de 1 columna
df[["col1", "col2"]]
df[inicio:fin] # slicing de filas por posición
df.loc[etiqueta_fila, etiqueta_col]
df.loc[[e1, e2], ["colA", "colB"]]
df.iloc[posicion_fila, posicion_col]
df.loc[:, ["col1", "col2"]] # todas las filas, columnas concretas
```

Ejemplo práctico:

```
serie_salario = empleados["salario_base"] # Serie
df_salario = empleados[["salario_base"]] # DataFrame

subset = empleados[["departamento", "ciudad", "salario_base"]]

filas_10_15 = empleados[10:16] # posiciones 10 a 15 incluidas

fila_emp = empleados.loc["EMP-1042"]

datos_varios = empleados.loc[["EMP-1005", "EMP-1020", "EMP-1099"], ["departamento", "modalidad"]]

primeras_tres = empleados.iloc[0:3, [2, 5]]

nombre_bonus = empleados.loc[:, ["nombre", "bonus_pct"]]
```

Forma	Trabaja por	Devuelve
<code>df["col"] / df[...]</code> <code>.loc[fila, col]</code>	nombre de columna etiqueta (nombre de índice/columna)	Serie o DataFrame valor, Serie o DataFrame
<code>.iloc[fila, col]</code>	posición numérica (0,1,2...)	valor, Serie o DataFrame

24. Crear columnas calculadas

Explicación: Se puede crear una columna nueva asignando el resultado de una operación entre columnas existentes; pandas opera de forma vectorizada, fila a fila, igual que NumPy.

Sintaxis básica:

```
df["nueva_col"] = df["col1"] * df["col2"]
df["nueva_col"] = (df["col1"] * (1 - df["col2"] / 100)).round(2)
```

Ejemplo práctico:

```
ventas["importe_bruto"] = ventas["unidades"] * ventas["precio_unitario"]
ventas["importe_netto"] = (ventas["importe_bruto"] * (1 - ventas["descuento_pct"] / 100)).round(2)
ventas["ticket_medio"] = ventas["importe_netto"] / ventas["unidades"]
```

25. Agrupación y agregación: `groupby()`, `.sum()`, `.agg()`, `.nlargest()`

Explicación: `groupby("columna")` agrupa filas que comparten el mismo valor en esa columna; se le encadena una función de agregación (`.sum()`, `.mean()`) para resumir cada grupo. `.agg({"col": "func"})` permite aplicar funciones distintas a columnas distintas en una sola llamada. `.nlargest(n, "columna")` devuelve las n filas con mayor valor en esa columna.

Sintaxis básica:

```
df.groupby("columna")["otra_columna"].sum().sort_values(ascending=False)
df.groupby(["col_a", "col_b"]).agg({"col_c": "mean", "col_d": "sum"})
df.groupby("columna")["valor"].sum().nlargest(n)
```

Ejemplo práctico:

```
por_region = ventas.groupby("region")["importe_netto"].sum().sort_values(ascending=False)

resumen_cat_canal = ventas.groupby(["categoria", "canal"]).agg({
    "importe_netto": "mean",
    "unidades": "sum",
})

top5_productos = ventas.groupby("producto")["importe_netto"].sum().nlargest(5)
```

26. Tratamiento de nulos: `.fillna()`, `.isna()`

Explicación: `.isna()` marca con True las celdas vacías (nulas). `.fillna(valor)` reemplaza los nulos por un valor fijo; se puede usar un estadístico como la mediana (`df["col"].median()`) para rellenos numéricos, o un texto para columnas categóricas.

Sintaxis básica:

```
df["col"].isna().sum()
df["col"] = df["col"].fillna(valor)
df["col"] = df["col"].fillna(df["col"].median())
```

Ejemplo práctico:

```
ventas["satisfaccion"] = ventas["satisfaccion"].fillna(ventas["satisfaccion"].median())
ventas["canal"] = ventas["canal"].fillna("Desconocido")
```

27. Exportar a CSV

Explicación: `.to_csv(ruta)` guarda un DataFrame como fichero CSV. `index=False` evita escribir el índice numérico como columna adicional en el fichero de salida (recomendable salvo que el índice tenga significado propio).

Sintaxis básica:

```
df.to_csv("ruta/salida.csv", index=False)
```

Ejemplo práctico:

```
por_region_df = por_region.reset_index()
por_region_df.to_csv("../outputs/facturacion_por_region.csv", index=False)
```

Sección F — Lógica y control de flujo

28. Operadores de comparación y valores booleanos

Explicación: `==` compara igualdad de **valor** (distinto de identidad); es sensible a mayúsculas en cadenas ("Norte" `==` "norte" es False). True/False se comportan como 1/0 en operaciones aritméticas. Las cadenas se comparan alfabéticamente (por orden Unicode) con `</>`. `!=` compara desigualdad; `3 != 3.0` es False porque Python compara el **valor numérico**, no el tipo.

Sintaxis básica:

```
a == b
a != b
a < b
```

Ejemplo práctico:

```
print(2 < 3)           # True
print("Norte" == "norte") # False, distinción de mayúsculas
print(True + True + False) # 2, True vale 1 y False vale 0
print(3 != 3.0)        # False, mismo valor numérico
print("Zapatilla" > "Mochila") # True, comparación alfabética por Unicode ('Z' > 'M')
```

29. Operadores lógicos en Python puro: and, or, not

Explicación: Combinan expresiones booleanas normales de Python (no arrays de NumPy, que usan `&/|`). `and` requiere que ambas condiciones sean verdaderas; `or` con que una lo sea; `not` invierte el resultado.

Sintaxis básica:

```
condicion1 and condicion2
condicion1 or condicion2
not condicion
```

Ejemplo práctico:

```
unidades = 6
precio = 149.0
descuento = 15

cumple_volumen_y_descuento = unidades > 4 and descuento >= 10 # True
precio_extremo = precio < 50 or precio > 200                    # False

no_cumple = not cumple_volumen_y_descuento # False: si era True, la negación es False
# En negocio: "no_cumple" indica si el pedido NO reúne las condiciones de volumen y descuento
```

Checklist rápido de conceptos por ejercicio

Ejercicio	Conceptos clave
1	slicing con paso, <code>.index()</code> , <code>sum()/len()/round()</code>
2	listas anidadas, doble indexación, límites del slicing como filtro
3	referencia vs copia (<code>=</code> , <code>list()</code> , <code>[:]</code>), <code>id()</code> , métodos en sitio
4	slicing con paso negativo, <code>sorted()</code> vs <code>.sort()</code> , <code>.pop()</code> , <code>.insert()</code>
5	parámetros por defecto, tuplas de retorno, <code>kwargs</code> , <code>docstring</code> , <code>help()</code>
6	<code>.strip()</code> , <code>.upper()</code> , <code>.split()</code> , <code>.join()</code> , <code>.count()</code> , <code>.replace()</code>
7	<code>import</code> , <code>from...import</code> , <code>alias</code> , <code>random.seed()</code> , módulo propio, <code>sys.path</code>
8	listas vs arrays, operaciones vectorizadas, <code>dtype</code> , <code>%timeit</code>
9	máscaras booleanas, <code>&</code> , <code>np.where()</code>
10	arrays 2D, <code>axis</code> , <code>np.vstack()</code> , <code>np.argmax()</code>
11	<code>default_rng</code> , <code>media/mediana/percentiles</code> , <code>np.corrcoef()</code>
12	diccionarios simples y anidados, <code>.items()</code> , <code>del</code> , <code>in</code>
13	<code>pd.DataFrame()</code> , <code>.set_index()</code> , <code>pd.read_csv()</code> , <code>.info()</code> , <code>.isna()</code>
14	<code>df["col"]</code> vs <code>df[["col"]]</code> , <code>.loc</code> , <code>.iloc</code>
15	columnas calculadas, <code>groupby()</code> , <code>.agg()</code> , <code>.nlargest()</code> , <code>.fillna()</code> , <code>.to_csv()</code>
16	operadores de comparación, <code>and/or/not</code> vs <code>&/ </code> en NumPy

Ejercicios resueltos

A continuación se resuelven los 16 ejercicios de la Práctica 3. Cada celda de código va precedida de una breve explicación, tal como exige el formato del notebook entregable.

Celda de verificación del entorno

```
import sys
import platform
import numpy as np
import pandas as pd

print("Python      :", sys.version)
print("Implementación:", platform.python_implementation())
print("NumPy       :", np.__version__)
print("pandas      :", pd.__version__)
```

```
assert sys.version_info[:2] == (3, 12), "Esta práctica requiere Python 3.12"
print("\nEntorno verificado correctamente.")
```

Ejercicio 1 — Catálogo de productos como lista

```
catalogo = [
    "Auriculares BT", 59.90,
    "Smartwatch S2", 149.00,
    "Tablet 10", 219.00,
    "Cargador USB-C", 18.50,
    "Cafetera Expres", 189.00,
    "Robot Aspirador", 279.00,
]

print("Número de elementos:", len(catalogo))
print("Número de productos:", len(catalogo) // 2)

nombres = catalogo[0::2]
precios = catalogo[1::2]
print("Nombres:", nombres)
print("Precios:", precios)

pos = nombres.index("Robot Aspirador")
print("Precio del Robot Aspirador:", precios[pos])

ultimos_tres_nombres = nombres[-3:]
print("Últimos tres productos:", ultimos_tres_nombres)

precio_medio = round(sum(precios) / len(precios), 2)
print("Precio medio del catálogo:", precio_medio)
```

Salida:

```
Número de elementos: 12
Número de productos: 6
Nombres: ['Auriculares BT', 'Smartwatch S2', 'Tablet 10', 'Cargador USB-C', 'Cafetera Expres', 'Robot Aspirador']
Precios: [59.9, 149.0, 219.0, 18.5, 189.0, 279.0]
Precio del Robot Aspirador: 279.0
Últimos tres productos: ['Cargador USB-C', 'Cafetera Expres', 'Robot Aspirador']
Precio medio del catálogo: 152.4
```

Comprobación: nombres tiene 6 elementos y sum(precios) = 914.40. (correcto)

Ejercicio 2 — Inventario multialmacén con listas anidadas

```
inventario = [
    ["ALM-NORTE", "Robot Aspirador", 34, 201.50],
    ["ALM-NORTE", "Monitor 27", 58, 142.00],
    ["ALM-SUR", "Silla Ergonomica", 12, 178.90],
    ["ALM-SUR", "Robot Aspirador", 7, 201.50],
    ["ALM-ESTE", "Cafetera Expres", 41, 131.20],
]
```

```

    ["ALM-ESTE", "Monitor 27", 25, 142.00],
]

print("Producto de la cuarta fila:", inventario[3][1])

valor_stock = []
for fila in inventario:
    almacen, producto, unidades, coste = fila
    valor_stock.append([producto, round(unidades * coste, 2)])
print("Valor por línea:", valor_stock)

valor_inmovilizado_total = round(sum(linea[1] for linea in valor_stock), 2)
print("Valor inmovilizado total:", valor_inmovilizado_total)

almacen_sur = inventario[2:4] # filas contiguas del ALM-SUR
valor_sur = round(sum(fila[2] * fila[3] for fila in almacen_sur), 2)
print("Valor ALM-SUR:", valor_sur)

```

Salida:

Producto de la cuarta fila: Robot Aspirador
 Valor por línea: [['Robot Aspirador', 6851.0], ['Monitor 27', 8236.0], ['Silla Ergonomica', 2146.8],
 Valor inmovilizado total: 27573.5
 Valor ALM-SUR: 3557.3

Comprobación: valor inmovilizado total = 27 573,50 €; ALM-SUR = 3 557,30 €. (correcto)

Documentación: El *slicing* solo funciona porque las filas del ALM-SUR están contiguas en la lista. Si un tercer almacén se insertara entre las dos filas de ALM-SUR, el *slicing* dejaría fuera una fila o incluiría filas de otro almacén, porque no filtra por contenido, sino por posición física; no es una estrategia robusta de filtrado.

Ejercicio 3 — Manipulación destructiva y copias de listas

```

precios_originales = [59.90, 149.00, 219.00, 18.50, 189.00, 279.00]

precios_rebajados = precios_originales
precios_rebajados[0] = 49.90
print("Original:", precios_originales) # también cambia
print("Rebajados:", precios_rebajados)
# Explicación: precios_rebajados apunta al mismo objeto que precios_originales,
# así que modificar uno modifica el otro.

precios_originales[0] = 59.90 # restaurar
copia_list = list(precios_originales)
copia_slice = precios_originales[:]
copia_list[0] = 0
copia_slice[1] = 0
print("Original tras modificar copias:", precios_originales) # sin cambios

copia_rebaja = list(precios_originales)
for i in range(len(copia_rebaja)):
    copia_rebaja[i] = round(copia_rebaja[i] * 0.9, 2)
print("Copia con 10% de rebaja:", copia_rebaja)

```

```

copia_rebaja.extend([99.90, 45.00])
copia_rebaja.insert(2, 129.00)
print("Tras extend e insert:", copia_rebaja)

del copia_rebaja[4]
descartado = copia_rebaja.pop()
print("Tras del y pop:", copia_rebaja, "| descartado:", descartado)

print(id(precios_originales) == id(copia_list))    # False
print(id(precios_originales) == id(copia_slice))  # False

```

Documentación: precios_rebajados = precios_originales no copia la lista, copia la **referencia**: ambas variables apuntan al mismo objeto en memoria, por lo que modificar una modifica la otra. list(...) y [:] crean objetos nuevos e independientes, confirmado porque id() devuelve valores distintos.

Ejercicio 4 — Cola de pedidos pendientes

```

cola = ["PED-10021", "PED-10007", "PED-10044", "PED-10012", "PED-10033", "PED-10008"]

cola_invertida = cola[::-1]
print("Cola invertida:", cola_invertida)

cola_ordenada = sorted(cola)
print("Ordenada (copia):", cola_ordenada)
print("Original tras sorted():", cola)    # sin cambios

cola.sort(reverse=True)
print("Cola tras .sort(reverse=True):", cola)

atendidos = [cola.pop(0), cola.pop(0)]
print("Atendidos:", atendidos)
print("Cola pendiente:", cola)

cola.insert(0, "PED-99999")
print("Cola con urgente:", cola)

total_inicial = len(atendidos) + len(cola)
porcentaje_avance = round(100 * len(atendidos) / total_inicial, 1)
print(f"Pedidos atendidos: {len(atendidos)}, pendientes: {len(cola)}, avance: {porcentaje_avance}")

```

Ejercicio 5 — Funciones propias para el cálculo de importes

```

def calcular_importe(unidades, precio_unitario, descuento_pct=0):
    """Devuelve el importe neto de una línea de pedido, redondeado a 2 decimales."""
    bruto = unidades * precio_unitario
    return round(bruto * (1 - descuento_pct / 100), 2)

def resumen_pedido(unidades, precio_unitario, descuento_pct=0):
    """Devuelve una tupla (importe_bruto, ahorro, importe_netto)."""

```

```

    bruto = round(unidades * precio_unitario, 2)
    neto = calcular_importe(unidades, precio_unitario, descuento_pct)
    ahorro = round(bruto - neto, 2)
    return bruto, ahorro, neto

bruto, ahorro, neto = resumen_pedido(unidades=3, precio_unitario=59.90, descuento_pct=10)
print(f"Bruto: {bruto}, Ahorro: {ahorro}, Neto: {neto}")

help(calcular_importe)

pedidos = [
    (3, 59.90, 10),
    (1, 279.00, 0),
    (5, 18.50, 20),
    (2, 149.00, 5),
    (4, 89.90, 15),
]
importes = [calcular_importe(u, p, d) for u, p, d in pedidos]

print("Importes:", importes)
print("Máximo:", max(importes))
print("Mínimo:", min(importes))
print("Ordenados:", sorted(importes))
print("Número de pedidos:", len(importes))

```

Comprobación: calcular_importe(3, 59.90, 10) devuelve 161.73. (correcto)

Ejercicio 6 — Normalización de códigos SKU con métodos

```

skus_crudos = [
    " elc-0012-es ", "HOG-0045-ES", "dep-0003-pt ",
    "ofi-0021-es", " ELC-0012-ES", "hog-0099-fr ",
]

skus_limpios = [s.strip().upper() for s in skus_crudos]
print("SKU limpios:", skus_limpios)

veces_elc0012 = skus_limpios.count("ELC-0012-ES")
print("Repeticiones de ELC-0012-ES:", veces_elc0012)

pos_ofi = skus_limpios.index("OFI-0021-ES")
print("Posición de OFI-0021-ES:", pos_ofi)

partes = [s.split("-") for s in skus_limpios]
print("Partes (familia, número, país):", partes)

skus_es = [s.replace("-", ".") for s in skus_limpios if s.endswith("ES")]
print("SKU de España:", skus_es)

cadena_unica = " | ".join(skus_es)
print("Cadena única:", cadena_unica)

```

Ejercicio 7 — Paquetes, importaciones y módulo propio

```
import math
raiz = math.sqrt(2809)
print("Raíz de 2809:", raiz)    # 53.0

from math import pi, ceil
palets = ceil(1375 / 24)
print("Palés necesarios para 1375 unidades:", palets)    # 58

import numpy as np
import pandas as pd
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)

import random
random.seed(42)
valoraciones = [random.randint(1, 5) for _ in range(10)]
print("Valoraciones:", valoraciones)
```

Explicación (una frase): Fijar la semilla con `random.seed(42)` hace que el generador pseudoaleatorio arranque siempre desde el mismo punto interno, así que la secuencia de números “aleatorios” que produce es idéntica cada vez, permitiendo que cualquiera que ejecute el mismo código obtenga exactamente el mismo resultado.

Fichero `src/utilidades.py`:

```
# src/utilidades.py
"""Funciones auxiliares de la Práctica 3."""

def calcular_importe(unidades, precio_unitario, descuento_pct=0):
    """Devuelve el importe neto de una línea de pedido."""
    bruto = unidades * precio_unitario
    return round(bruto * (1 - descuento_pct / 100), 2)

def clasificar_ticket(importe):
    """Clasifica un importe en 'Bajo', 'Medio' o 'Alto'."""
    if importe < 100:
        return "Bajo"
    elif importe < 400:
        return "Medio"
    return "Alto"
```

Uso desde el notebook:

```
import sys
sys.path.append("../src")

from utilidades import calcular_importe, clasificar_ticket

print(calcular_importe(4, 89.90, 15))    # 305.66
print(clasificar_ticket(305.66))         # "Medio"
```

Comprobación: `calcular_importe(4, 89.90, 15) = 305.66`; `clasificar_ticket(305.66) = "Medio"`. (correcto)

Ejercicio 8 — Del cálculo con listas al cálculo vectorizado

```
import numpy as np

unidades = [3, 1, 5, 2, 4, 6, 2, 8, 3, 1]
precios = [59.90, 279.00, 18.50, 149.00, 89.90, 27.50, 219.00, 14.90, 64.00, 239.00]

# unidades * precios con listas de Python no multiplica elemento a elemento:
# repite la lista 'unidades' tantas veces como indique el primer elemento de 'precios'
# si se intenta directamente (o lanza TypeError según el caso), no es la operación matemática esp

np_unidades = np.array(unidades)
np_precios = np.array(precios)
print("dtype unidades:", np_unidades.dtype, "shape:", np_unidades.shape)
print("dtype precios:", np_precios.dtype, "shape:", np_precios.shape)

importes = np_unidades * np_precios
print("Importes:", importes)
print("Suma total:", importes.sum())

importes_con_recargo = importes * 1.04
print("Con recargo del 4%:", importes_con_recargo)

mixto = np.array([1, "dos", 3.0, True])
print("dtype del array mixto:", mixto.dtype)
# NumPy convierte todos los elementos al tipo más general para mantener el array homogéneo;
# como hay texto, todos los valores se convierten a cadena.

# %timeit para comparar bucle for vs vectorizado (celda mágica de Jupyter):
# %timeit [u * p for u, p in zip(unidades, precios)]
# %timeit np_unidades * np_precios
```

Comprobación: importes.sum() = 2362.00 €; productos con precio > 100 €: 4 (279.00, 149.00, 89.90 no supera 100... revisar con máscara en el ejercicio 9). (correcto) (la comprobación exacta de “4 productos” se confirma con la máscara booleana del Ejercicio 9).

Ejercicio 9 — Subsetting con máscaras booleanas

```
caros = np_precios > 100
print("Máscara caros:", caros)

precios_caros = np_precios[caros]
print("Precios > 100:", precios_caros)

num_caros = caros.sum()
print("Número de productos caros:", num_caros)
# Sumar un array booleano funciona porque True equivale a 1 y False a 0;
# la suma cuenta cuántos elementos cumplen la condición.

importes_caros = importes[caros]
print("Importes de esas líneas:", importes_caros, "Suma:", importes_caros.sum())

posiciones = np.where(caros)
```

```
print("Posiciones:", posiciones)

varias = np_unidades >= 2
seleccion = (np_precios > 100) & (varias)
print("Importes con precio>100 y unidades>=2:", importes[seleccion])
```

Comprobación: hay 4 productos con precio > 100 € y la suma de sus importes es 1 254,00 €. (correcto)

Ejercicio 10 — Matriz de ventas por trimestre y región

```
import numpy as np

ventas_2d = np.array([
    [15200.50, 11800.25, 12950.00, 9870.40, 13400.10],
    [16840.75, 12310.60, 13480.90, 11020.35, 14100.80],
    [12535.72, 10663.06, 11221.59, 12673.08, 12334.38],
])

print("shape:", ventas_2d.shape)    # (3, 5): 3 trimestres, 5 regiones
print("ndim:", ventas_2d.ndim)      # 2: es una matriz (filas y columnas)
print("size:", ventas_2d.size)      # 15: total de celdas (trimestre x región)

fila_t2 = ventas_2d[1, :]
columna_centro = ventas_2d[:, 4]
print("T2:", fila_t2)
print("Región Centro:", columna_centro)

valor_t3_este = ventas_2d[2, 2]
print("T3 en Este:", valor_t3_este)

submatriz = ventas_2d[0:2, 0:3]
print("Submatriz T1-T2, tres primeras regiones:\n", submatriz)

total_por_trimestre = ventas_2d.sum(axis=1)
total_por_region = ventas_2d.sum(axis=0)
print("Total por trimestre:", total_por_trimestre)
print("Total por región:", total_por_region)

prevision_t4 = ventas_2d[2, :] * 1.06
ventas_ampliada = np.vstack([ventas_2d, prevision_t4])
print("Con previsión T4:\n", ventas_ampliada)

regiones = ["Norte", "Sur", "Este", "Oeste", "Centro"]
region_lider = regiones[np.argmax(total_por_region)]
print("Región con mayor facturación acumulada:", region_lider)
```

Ejercicio 11 — Estadística descriptiva sobre telemetría simulada

```
import numpy as np
```

```

rng = np.random.default_rng(2024)
temperatura = rng.normal(68, 6.5, 500)
vibracion = np.abs(rng.normal(2.4, 0.8, 500))

media = temperatura.mean()
mediana = np.median(temperatura)
desviacion = temperatura.std()
print("Media:", round(media, 2), "Mediana:", round(mediana, 2),
      "Desv. típica:", round(desviacion, 2),
      "Min:", round(temperatura.min(), 2), "Max:", round(temperatura.max(), 2))
# Media y mediana son muy similares porque los datos se generaron con una
# distribución normal (simétrica), donde ambas medidas de tendencia central coinciden.

percentiles = np.percentile(vibracion, [25, 50, 75, 95])
print("Percentiles 25/50/75/95 de vibración:", percentiles)

matriz_2d = np.column_stack([temperatura, vibracion])
print("shape matriz combinada:", matriz_2d.shape) # (500, 2)

correlacion = np.corrcoef(temperatura, vibracion)
print("Matriz de correlación:\n", correlacion)
# El coeficiente fuera de la diagonal está cerca de 0, indicando que no existe
# relación lineal apreciable entre temperatura y vibración en estos datos simulados
# (se generaron de forma independiente).

umbral = media + 2 * desviacion
n_superan = (temperatura > umbral).sum()
pct_superan = round(100 * n_superan / len(temperatura), 2)
print(f"Lecturas por encima de {round(umbral, 2)}: {n_superan} ({pct_superan}%)")

```

Explicación (dos frases): Fijar la semilla con `default_rng(2024)` garantiza que la secuencia de números generados sea siempre la misma para quien ejecute este código, incluido el profesor. Sin una semilla fija, cada ejecución produciría datos distintos y sería imposible comparar resultados o verificar que el análisis es correcto.

Ejercicio 12 — Diccionarios simples y anidados

```

tarifas_envio = {"Norte": 4.95, "Sur": 5.50, "Este": 5.20, "Oeste": 6.10, "Centro": 3.90}

print("Tarifa Este:", tarifas_envio["Este"])

tarifas_envio["Insular"] = 9.80
tarifas_envio["Oeste"] = 5.95
print("Tras añadir/actualizar:", tarifas_envio)

del tarifas_envio["Sur"]
print("¿Sigue 'Sur'? ", "Sur" in tarifas_envio)

for region, tarifa in tarifas_envio.items():
    print(f"{region} -> {tarifa} €")

red_logistica = {
    "Norte": {"almacen": "ALM-NORTE", "tarifa": 4.95, "entrega_h": 24},

```

```

    "Este": {"almacen": "ALM-ESTE", "tarifa": 5.20, "entrega_h": 48},
    "Centro": {"almacen": "ALM-CENTRO", "tarifa": 3.90, "entrega_h": 24},
}

print("Entrega Este:", red_logistica["Este"]["entrega_h"])

red_logistica["Oeste"] = {"almacen": "ALM-OESTE", "tarifa": 5.95, "entrega_h": 24}

for region, datos in red_logistica.items():
    print(f"{region}:")
    for campo, valor in datos.items():
        print(f"    {campo} -> {valor}")

```

Explicación (dos frases): Un diccionario conviene cuando se necesita acceder a un dato por su nombre o significado (por ejemplo, la tarifa de “Este”) en lugar de por su posición, lo que hace el acceso más directo y el código más legible. Una lista, en cambio, es preferible cuando el orden de los elementos importa y no hay un identificador natural que sirva de clave.

Ejercicio 13 — De diccionario a DataFrame y lectura de CSV

```

import pandas as pd

almacenes = pd.DataFrame({
    "codigo": ["ALM-NORTE", "ALM-SUR", "ALM-ESTE", "ALM-OESTE", "ALM-CENTRO"],
    "ciudad": ["Bilbao", "Sevilla", "Valencia", "Vigo", "Madrid"],
    "m2": [4200, 3100, 3800, 2600, 6500],
    "operarios": [48, 31, 39, 22, 74],
})
print(almacenes)

almacenes_idx = almacenes.set_index("codigo")
print(almacenes_idx)

ventas = pd.read_csv("../data/ventas_retail.csv")
empleados = pd.read_csv("../data/empleados.csv", index_col="id_empleado")
sensores = pd.read_csv("../data/sensores_planta.csv")

print(ventas.head(8))
print(ventas.tail(3))
print("shape ventas:", ventas.shape)

ventas.info()

nulos = ventas.isna().sum()
print(nulos)
# Los nulos se concentran en 'canal' y 'satisfaccion', tal como se documenta
# en la descripción de los datos de partida.

print("shape ventas:", ventas.shape)      # (420, 10)
print("shape empleados:", empleados.shape) # (180, 8)
print("shape sensores:", sensores.shape)   # (600, 8)

```

Comprobación: ventas 420 filas, empleados 180 filas, sensores 600 filas. (correcto)

Ejercicio 14 — Selección con corchetes, loc e iloc

```
serie_salario = empleados["salario_base"]
df_salario = empleados[["salario_base"]]
print(type(serie_salario), type(df_salario))    # <class 'pandas.core.series.Series'> <class 'pandas.core.frame.DataFrame'>

subset = empleados[["departamento", "ciudad", "salario_base"]]
print(subset.head())

filas_10_15 = empleados[10:16]
print(filas_10_15)

fila_emp = empleados.loc["EMP-1042"]
print(fila_emp)

datos_varios = empleados.loc[["EMP-1005", "EMP-1020", "EMP-1099"], ["departamento", "modalidad"]]
print(datos_varios)

primeras_tres = empleados.iloc[0:3, [2, 5]]
print(primeras_tres)

nombre_bonus = empleados.loc[:, ["nombre", "bonus_pct"]]
print(nombre_bonus.head())
```

Forma	Trabaja por	Ejemplo
Corchetes <code>df["col"]</code> / <code>df[a:b]</code>	nombre de columna o posición de fila	<code>empleados["salario_base"]</code> , <code>empleados[10:16]</code>
<code>.loc[fila, col]</code>	etiqueta (nombre de índice y de columna)	<code>empleados.loc["EMP-1042", "departamento"]</code>
<code>.iloc[fila, col]</code>	posición numérica (0,1,2...)	<code>empleados.iloc[0:3, [2, 5]]</code>

Ejercicio 15 — Columnas calculadas, agregación y exportación

```
ventas["importe_bruto"] = ventas["unidades"] * ventas["precio_unitario"]
ventas["importe_netto"] = (ventas["importe_bruto"] * (1 - ventas["descuento_pct"] / 100)).round(2)
ventas["ticket_medio"] = ventas["importe_netto"] / ventas["unidades"]

por_region = ventas.groupby("region")["importe_netto"].sum().sort_values(ascending=False)
print(por_region)

resumen_cat_canal = ventas.groupby(["categoria", "canal"]).agg({
    "importe_netto": "mean",
    "unidades": "sum",
})
print(resumen_cat_canal)

top5_productos = ventas.groupby("producto")["importe_netto"].sum().nlargest(5)
print(top5_productos)
```

```

ventas["satisfaccion"] = ventas["satisfaccion"].fillna(ventas["satisfaccion"].median())
ventas["canal"] = ventas["canal"].fillna("Desconocido")

por_region_df = por_region.reset_index()
por_region_df.to_csv("../outputs/facturacion_por_region.csv", index=False)

print("Facturación neta total:", round(ventas["importe_neto"].sum(), 2))
print("Región líder:", por_region.idxmax())

```

Comprobación: facturación neta total = 190 402,48 €; región líder = **Norte**. (correcto)

Ejercicio 16 — Operadores de comparación y booleanos

```

print(2 < 3)                # True: 2 es efectivamente menor que 3
print("Norte" == "norte")   # False: la comparación de cadenas distingue mayúsculas/minúsculas
print(True + True + False)  # 2: True vale 1 y False vale 0 en aritmética
print(3 != 3.0)             # False: Python compara el valor numérico, 3 y 3.0 son iguales
print("Zapatilla" > "Mochila") # True: comparación alfabética por código Unicode ('Z' > 'M')

unidades = 6
precio = 149.0
descuento = 15

cumple_volumen_y_descuento = unidades > 4 and descuento >= 10
print("Cumple volumen y descuento:", cumple_volumen_y_descuento) # True

precio_extremo = precio < 50 or precio > 200
print("Precio extremo:", precio_extremo) # False, 149 no es ni muy bajo ni muy alto

no_cumple = not cumple_volumen_y_descuento
print("Negación:", no_cumple)
# En términos de negocio: no_cumple indica si el pedido NO reúne, a la vez,
# más de 4 unidades y un descuento de al menos el 10%; aquí es False porque sí las reúne.

import numpy as np
temp = np.array([64.2, 88.1, 71.0, 92.5, 69.8])
vib = np.array([1.9, 3.4, 2.1, 4.0, 2.2])

# temp > 70 and vib > 3
# ValueError: The truth value of an array with more than one element is ambiguous.
# 'and'/'or' esperan un único valor booleano, pero 'temp > 70' y 'vib > 3' son
# arrays con varios True/False; Python no sabe reducirlos a un solo booleano.
# La forma correcta sería: (temp > 70) & (vib > 3)

```